

FOUNDATION_KAFKA - Event Bus Foundation

Developer implementation guide: FOUNDATION_KAFKA_DEV_IMPL_GUIDE.md.

1. Purpose

This foundation defines how SOPHIX modules publish and consume business events using Kafka.

Use this document when implementing:

- A Kafka topic.
- A domain event.
- Producer outbox logic.
- Consumer inbox logic.
- Replay and reconciliation endpoints.
- Cache invalidation consumers.

2. Ownership

FOUNDATION_KAFKA owns:

- Kafka cluster topology.
- Topic naming.
- Message key format.
- Shared event envelope.
- Producer outbox rules.
- Consumer inbox rules.
- Replay and reconciliation mechanics.
- Kafka resilience expectations.

FOUNDATION_KAFKA does not own:

- Module-specific event payload fields. Each DD owns its payload.
- Database table placement. See FOUNDATION_DB.md.
- Cache update logic. See FOUNDATION_CACHE.md.

3. Cluster Topology

Item	Requirement
Kafka nodes	4
Mode	KRaft, no ZooKeeper
Broker role	All 4 nodes
Controller role	3 of the 4 nodes only, to keep an odd KRaft controller quorum
Replication factor	3
min.insync.replicas	2
Producer acks	acks=all
Default partitions	6 per topic
Retention	7 days
Auth	SASL/PLAIN over TLS
Bootstrap	kafka-1.sophix.internal:9092, kafka-2.sophix.internal:9092, kafka-3.sophix.internal:9092, kafka-4.sophix.internal:9092

Kafka is not behind NGINX. Clients connect to the broker bootstrap list directly.

Availability rule: this topology is designed to survive one broker failure cleanly. Two broker failures may make some partitions read-only or unavailable depending on replica placement. If the operator requires guaranteed write availability during any two broker failures, increase to a larger Kafka topology and revisit replication factor/min-ISR.

4. Topic Naming

Pattern:

```
sophix.{module}.{aggregate}.{event}
```

Examples:

```
sophix.plm.service.created  
sophix.billing.invoice.issued  
sophix.subscription.subscription.activated  
sophix.workorders.workorder.finalized
```

Rules:

Rule	Description
KAFKA-TOPIC-1	Topic names are lowercase.
KAFKA-TOPIC-2	Use singular aggregate names unless the owning DD already defines a canonical aggregate.
KAFKA-TOPIC-3	Event suffix is past tense for completed facts: created, issued, activated.
KAFKA-TOPIC-4	Do not publish command/request messages on domain event topics.

5. Message Key

Pattern:

```
{module}:{aggregate}:{id}
```

Examples:

```
plm:service:svc_01HX2Y3Z4  
billing:invoice:inv_01HX2Y3Z4  
subscription:subscription:sub_01HX2Y3Z4
```

All events for the same aggregate must use the same key. Kafka uses the key to keep aggregate events on the same partition and preserve per-aggregate order.

6. Event Envelope

Every event must use this envelope:

```
{  
  "eventId": "billing:invoice:inv_01HX:evt_01HY",  
  "eventType": "InvoiceIssued",  
  "aggregateId": "inv_01HX",  
  "timestamp": "2026-05-11T10:23:00Z",  
  "actor": "billing-service",  
  "correlationId": "corr_01HX",  
  "payload": {}  
}
```

Field rules:

Field	Rule
eventId	Globally unique event ID. Format: {messageKey}:evt_{ulid}.
eventType	PascalCase business event name.
aggregateId	ID of the aggregate that produced the event.
timestamp	UTC ISO-8601 timestamp.
actor	User, service account, or system.
correlationId	Propagated request/workflow correlation ID.
payload	DD-owned JSON body.

Do not reuse the Kafka message key as eventId. One aggregate emits many events over its lifetime.

7. Producer Outbox Pattern

Every business event must be written to a producer outbox table in the same transaction as the business write.

```
sequenceDiagram
    participant API as Module API
    participant DB as Module DB
    participant Worker as Outbox Worker
    participant Kafka as Kafka

    API->>DB: Insert/update business row
    API->>DB: Insert messaging.outbox_event
    API->>DB: Commit
    Worker->>DB: Read unpublished rows
    Worker->>Kafka: Publish with acks=all
    Worker->>DB: Mark published_at
```

Producer rules:

Rule	Description
KAFKA-PROD-1	Business write and outbox insert must be in the same DB transaction.
KAFKA-PROD-2	Outbox worker publishes only committed outbox rows.
KAFKA-PROD-3	Producer config must use acks=all.
KAFKA-PROD-4	Mark published_at only after Kafka publish succeeds.
KAFKA-PROD-5	Keep published outbox rows for 30 days.

8. Consumer Inbox Pattern

Every consumer must store incoming events in an inbox before processing.

```
sequenceDiagram
    participant Kafka as Kafka
    participant Poller as Consumer Poller
    participant DB as Consumer DB
    participant Worker as Inbox Worker

    Poller->>Kafka: Poll message
    Poller->>DB: Insert inbox row on eventId, ignore duplicate
    Poller->>Kafka: Commit offset
    Worker->>DB: Read unprocessed inbox rows
    Worker->>Worker: Handle event idempotently
    Worker->>DB: Mark processed_at
```

Consumer rules:

Rule	Description
KAFKA-CONS-1	eventId is the inbox primary key.
KAFKA-CONS-2	Duplicate events must be ignored by inbox insert conflict handling.
KAFKA-CONS-3	Kafka offset is committed after inbox insert, not after business processing.
KAFKA-CONS-4	Business processing is driven from inbox rows.
KAFKA-CONS-5	Failed processing increments attempts and stores last_error.
KAFKA-CONS-6	Inbox rows remain retryable until processed or manually resolved.
KAFKA-CONS-7	Keep processed inbox rows for 7 days.

9. Required Tables

Producer outbox:

```
CREATE TABLE messaging.outbox_event (
    event_id      TEXT PRIMARY KEY,
    topic         TEXT NOT NULL,
    message_key   TEXT NOT NULL,
    envelope      JSONB NOT NULL,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    published_at  TIMESTAMPTZ NULL,
    attempts      INTEGER NOT NULL DEFAULT 0,
    last_error    TEXT NULL
);
```

Consumer inbox:

```
CREATE TABLE messaging.inbox_event (  
  event_id      TEXT PRIMARY KEY,  
  topic         TEXT NOT NULL,  
  message_key   TEXT NOT NULL,  
  envelope      JSONB NOT NULL,  
  received_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  processed_at  TIMESTAMPTZ NULL,  
  attempts      INTEGER NOT NULL DEFAULT 0,  
  last_error    TEXT NULL  
);
```

Indexes:

```
CREATE INDEX idx_outbox_pending  
ON messaging.outbox_event (created_at)  
WHERE published_at IS NULL;  
  
CREATE INDEX idx_inbox_pending  
ON messaging.inbox_event (received_at)  
WHERE processed_at IS NULL;
```

10. Producer Example

```
BEGIN;  
  
INSERT INTO service (id, code, created_by, updated_by)  
VALUES ('svc_01HX', 'GPON_INTERNET_100M', 'alice', 'alice');  
  
INSERT INTO messaging.outbox_event (  
  event_id,  
  topic,  
  message_key,  
  envelope  
) VALUES (  
  'plm:service:svc_01HX:evt_01HY',  
  'sophix.plm.service.created',  
  'plm:service:svc_01HX',  
  '{  
    "eventId": "plm:service:svc_01HX:evt_01HY",  
    "eventType": "ServiceCreated",  
    "aggregateId": "svc_01HX",  
    "timestamp": "2026-05-11T10:23:00Z",  
    "actor": "alice",  
    "correlationId": "corr_01HX",  
    "payload": {  
      "serviceId": "svc_01HX",  
      "code": "GPON_INTERNET_100M"  
    }  
  }'::jsonb  
);  
  
COMMIT;
```

11. Replay

Each producer module must expose:

```
POST /admin/outbox/replay  
Authorization: Bearer <admin-jwt>  
Content-Type: application/json  
  
{  
  "topic": "sophix.plm.service.created",  
  "fromTime": "2026-05-09T00:00:00Z",  
  "toTime": "2026-05-10T00:00:00Z",  
  "targetConsumerGroup": "subscription-engine"  
}
```

Replay behavior:

1. Read matching rows from producer outbox.
2. Publish to replay topic:

```
{original-topic}.replay
```
3. Target consumer temporarily subscribes to replay topic.
4. Consumer inbox deduplicates by original eventId.

5. Run reconciliation after replay.

12. Monitoring

Alert on:

Signal	Threshold
Unpublished outbox rows older than 1 minute	More than 0 sustained
Unprocessed inbox rows older than 5 minutes	More than 0 sustained
Inbox rows with attempts > 10	More than 0
Consumer group lag	More than 1000 messages
Broker count	Less than 4
Broker disk usage	More than 70 percent

13. Failure Behavior

Failure	Behavior
One broker down	Cluster continues. Producers and consumers reconnect.
Two brokers down	Some partitions may lose the required two in-sync replicas. Restore brokers urgently. Do not plan maintenance with one broker already down.
Three or more brokers down	Producers cannot publish. Outbox accumulates. No committed business event is lost.
Consumer down	Kafka retains messages. Consumer resumes by offset. Inbox deduplicates.
Consumer bug causes drift	Replay from producer outbox and reconcile.
Kafka retention exceeded	Replay from outbox if within 30-day outbox retention.

14. Medium Budget Infrastructure

This section defines the recommended on-premise starting point for a medium SOPHIX deployment.

14.1 Server Dimensions

Required medium baseline:

Server role	Count	CPU	RAM	Disk	Notes
Kafka node	4	4 vCPU each	16 GB each	1 TB SSD each	All 4 run broker role. Three also run KRaft controller role.
Monitoring/log storage	Shared platform service	2-4 vCPU	8 GB	Depends on retention	Needed for broker logs, lag metrics, alerts.

Preferred medium baseline:

4 x Kafka nodes: 4-8 vCPU / 16-32 GB RAM / 1-2 TB SSD

Budget rule: keep four Kafka nodes. Reduce CPU/RAM before reducing node count. A three-node Kafka cluster is possible, but it gives less broker headroom and less operational flexibility.

Placement rules:

- Spread Kafka nodes across physical hosts.
- Do not place all KRaft controller-role nodes on the same physical host.
- Use SSD. Kafka is disk and network sensitive.
- Keep Kafka private. Do not expose broker listeners publicly.
- Keep Kafka and PostgreSQL on separate servers. Do not co-locate Kafka brokers with DB primaries.

14.2 Recommended OS

Use one operator-standard Linux distribution:

OS	Recommendation
Ubuntu Server 22.04 LTS or 24.04 LTS	Recommended default for medium teams.
Rocky Linux 9 / RHEL 9	Use if the operator standardizes on RHEL-family systems.

OS baseline:

- 64-bit minimal server install.
- No desktop packages.
- chrony or equivalent NTP enabled.
- Security patching enabled through operator maintenance process.
- SSH restricted to admin VPN/network.
- Host firewall enabled.
- Java runtime version pinned and tested with the selected Kafka version.

14.3 Disk Partitions

For each Kafka node with 1 TB disk:

```

/                40 GB
/opt/kafka       20 GB
/var/lib/kafka   820 GB
/var/log         40 GB
/tmp            20 GB
free/reserved    remaining

```

For each Kafka node with 2 TB disk:

```

/                40 GB
/opt/kafka       20 GB
/var/lib/kafka   1750 GB
/var/log         60 GB
/tmp            30 GB
free/reserved    remaining

```

Partition rules:

- Put Kafka log segments on `/var/lib/kafka`.
- Do not store Kafka data on the root filesystem.
- Alert at 70 percent disk usage and treat 85 percent as urgent.
- Keep enough free space for partition reassignment and broker recovery.
- Use `xfs` or `ext4`.

14.4 JVM and Kafka Runtime Starting Points

For 16 GB RAM nodes:

```
KAFKA_HEAP_OPTS="-Xms4g -Xmx4g"
```

For 32 GB RAM nodes:

```
KAFKA_HEAP_OPTS="-Xms6g -Xmx6g"
```

Starting broker defaults:

```

num.partitions=6
default.replication.factor=3
min.insync.replicas=2
log.retention.hours=168
unclean.leader.election.enable=false
auto.create.topics.enable=false

```

Topic creation should be explicit and reviewed. Disable auto topic creation in production.

14.5 Network Ports

Flow	Port	Exposure
Client to broker	9092 or 9094	Internal application network only

Flow	Port	Exposure
Broker-to-broker replication	Kafka listener port	Kafka network only
KRaft controller quorum	9093 or operator-selected	Kafka nodes only
Admin SSH	22	Admin VPN/network only
Monitoring scrape	Operator standard	Monitoring network only

15. AWS Deployment Recommendations

This section applies only when deploying the event bus on AWS. Prefer Amazon MSK unless there is a strong reason to self-manage Kafka.

15.1 Recommended AWS Shape

Component	AWS service	Recommendation
Kafka cluster	Amazon MSK Provisioned	Recommended default for production.
Serverless option	Amazon MSK Serverless	Consider only if workload is highly variable and regional availability/cost model fits.
Broker subnets	Private subnets	Spread brokers across at least 2 Availability Zones, preferably 3 if region and cost allow.
Auth	IAM/SASL/SCRAM/TLS depending on client support	Standardize per environment.
Logs/metrics	CloudWatch	Enable broker logs and metrics.

Do not expose Kafka publicly. Application services should connect from private subnets/security groups.

15.2 MSK Sizing

Medium budget start:

```
MSK Provisioned:
  Brokers:      4 if supported by the chosen AZ layout; otherwise 3 brokers across 3 AZs
  Broker type:  kafka.m7g.large or kafka.m5.large
  Storage:      1 TB per broker to start
  Replication:  topic RF=3
  minISR:      2
```

Preferred medium:

```
MSK Provisioned:
  Brokers:      6 across 3 AZs
  Broker type:  kafka.m7g.xlarge or larger
  Storage:      1-2 TB per broker
  Replication:  topic RF=3
  minISR:      2
```

Use Graviton broker classes such as `kafka.m7g.*` where supported in the selected region and by client/security requirements. Use `kafka.m5.*` where the team needs x86 compatibility.

15.3 MSK Serverless Guidance

MSK Serverless can reduce capacity planning work, but use it only after confirming:

- The target AWS Region supports MSK Serverless.
- Required authentication mode is supported by SOPHIX clients.
- Throughput-based pricing is acceptable for expected event volume.
- Operational tooling supports the serverless cluster type.

For core BSS production, MSK Provisioned is the safer default because capacity, broker count, and storage are explicit.

15.4 AWS Network Layout

Recommended VPC layout:

```
Private app subnets:
  - SOPHIX services
```

```
Private streaming subnets:
  - MSK brokers
```

- Public subnets:
- No Kafka resources

Security group rules:

Source	Destination	Port	Purpose
App service SG	MSK broker SG	MSK client listener	Produce/consume
Admin VPN/bastion/SSM	Client tools host	SSH/SSM	Admin tools
Monitoring service	MSK/CloudWatch	AWS-managed	Metrics and alarms

15.5 AWS Monitoring

Alarm on:

Signal	Threshold
Offline partitions	More than 0
Under-replicated partitions	More than 0 sustained
Active controller changes	Unexpected changes
Broker disk usage	Above 70 percent
Consumer lag	Above module-specific threshold
Produce/request error rate	Any sustained increase
Network throughput	Near broker limit

Reference docs:

- [Amazon MSK documentation](https://aws.amazon.com/documentation-overview/msk/)
- [Amazon MSK broker sizes](https://docs.aws.amazon.com/msk/latest/developerguide/broker-instance-sizes.html)
- [Amazon MSK Serverless](https://docs.aws.amazon.com/msk/latest/developerguide/serverless.html)

16. Operational Endpoints

Each module with Kafka integration must implement:

Endpoint	Purpose
GET /admin/outbox/pending	Show unpublished outbox rows
POST /admin/outbox/replay	Replay historical events
GET /admin/inbox/pending	Show unprocessed inbox rows
POST /admin/inbox/retry/{eventId}	Reset a stuck inbox row
GET /admin/reconcile?aggregate={type}&since={ts}	Compare consumer view with source-of-truth

17. Developer Checklist

- [] Topic follows `sophix.{module}.{aggregate}.{event}`.
- [] Key follows `{module}:{aggregate}:{id}`.
- [] Event envelope includes all required fields.
- [] Event payload is additive-version safe.
- [] Producer writes business row and outbox row in one transaction.
- [] Producer uses `acks=all`.
- [] Consumer writes inbox row before processing.
- [] Handler is idempotent.
- [] Reprocessing same `eventId` is safe.
- [] Admin replay/reconcile endpoints exist for critical consumers.